

Acceso a Datos - Semana 3

UD1. Manejo de ficheros - Ficheros de texto y CSV

Objetivo de la semana: leer y escribir información persistente en ficheros de texto, trabajar con codificación, gestionar recursos correctamente y procesar datos en formato CSV.

1. Recordatorio: ¿qué es un fichero de texto?

Un fichero de texto almacena información como una secuencia de caracteres. Su contenido puede abrirse con un editor de texto y, a diferencia de un fichero binario, los datos son legibles directamente por una persona.

Ejemplo de contenido de un fichero alumnos.txt:

```
1;Ana;DAM
2;Pedro;DAW
3;Lucía;DAM
```

El programa Java no ve inicialmente esos datos como objetos Alumno. Primero debe leer los caracteres y después interpretar la estructura del contenido.

2. Flujo de entrada y flujo de salida

En Java se trabaja con la idea de flujo (stream). Un flujo representa una secuencia de datos que entra o sale del programa.

Entrada	Salida
Fichero -> programa	Programa -> fichero
Leer	Escribir

Para ficheros de texto veremos dos familias clásicas: FileReader/BufferedReader para lectura y FileWriter/BufferedWriter para escritura. También veremos la alternativa moderna con Files.

3. Metodos FileReader y FileWriter

Método/Constructor	Descripción
FileReader(String fileName)	Crea un objeto FileReader asociado con un archivo cuyo nombre se especifica como argumento.
int read()	Lee un único carácter del archivo y devuelve un entero que representa el valor Unicode del carácter leído o -1 si se alcanza el final del archivo.
FileWriter(String fileName)	crea un objeto FileWriter asociado con un archivo cuyo nombre se especifica como argumento.
FileWriter(String fileName, boolean append)	Crea un objeto FileWriter asociado con un archivo y permite especificar si se desea agregar contenido al final del archivo existente.
void write(int c)	Escribe un único carácter en el archivo.
void write(String str)	Escribe una cadena de caracteres en el archivo.
void close()	Cierra el flujo y libera los recursos asociados.

4. Lectura con FileReader

FileReader permite leer caracteres. Puede utilizarse directamente, aunque para leer líneas completas suele combinarse con BufferedReader.

```
try (FileReader fr = new FileReader("datos.txt")) {  
  
    int caracter;  
  
    while ((caracter = fr.read()) != -1) {  
        System.out.print((char) caracter);  
    }  
  
} catch (IOException e) {  
    System.out.println("Error: " + e.getMessage());  
}
```

read() devuelve un entero, que corresponde con el valor UNICODE del caracter. El valor -1 indica que se ha llegado al final del fichero.

```
String fichero = "fichero.txt";  
  
try {  
    // creamos un objeto FileReader  
    FileReader fileReader = new FileReader(fichero);  
  
    // leer y mostrar el contenido del archivo  
    int caracter;  
    System.out.println("Contenido del archivo " + fichero + ":");  
    while ((caracter = fileReader.read()) != -1) {  
        System.out.print((char) caracter);  
    }  
  
    // cerramos el FileReader  
    fileReader.close();  
} catch (IOException e) {  
    System.out.println("Ocurrió un error al leer el archivo");  
    e.printStackTrace();  
}
```

5. Escritura con FileWriter

FileWriter permite escribir caracteres en un fichero de texto.

```
import java.io.FileWriter;  
import java.io.IOException;  
  
public class Ejemplo {  
    public static void main(String[] args) {  
        try {  
            FileWriter fw = new FileWriter("datos.txt");  
            fw.write("Hola desde Java");  
            fw.close();  
        } catch (IOException e) {  
            System.out.println("Error: " + e.getMessage());  
        }  
    }  
}
```

```

    }
}

```

Si el fichero no existe, normalmente se crea. Si ya existe, esta forma de abrir `FileWriter` reemplaza su contenido.

```

String texto = "Este es un ejemplo de uso de FileWriter en JAVA";
String fichero = "fichero.txt";

try {
    // creamos un objeto FileWriter
    FileWriter fileWriter = new FileWriter(fichero);

    // escribimos una String en el archivo
    fileWriter.write(texto);

    // cerramos el FileWriter
    fileWriter.close();

    System.out.println("Se ha escrito en el fichero correctamente");
} catch (IOException e) {
    System.out.println("Ocurrió un error al escribir en el fichero");
    e.printStackTrace();
}

```

6. Añadir contenido sin borrar lo anterior

El segundo parámetro del constructor de `FileWriter` permite activar el modo append (añadir al final).

```

FileWriter fw = new FileWriter("datos.txt", true);
fw.write("Nueva línea\n");
fw.close();

```

Con `true` se conserva el contenido anterior. Con `false` (o sin segundo parámetro), se sobrescribe.

7. Métodos `BufferedReader` y `BufferedWriter`

Método/Constructor	Descripción
<code>String readLine()</code>	Lee una línea de texto
<code>int read()</code>	Lee un único carácter del archivo y devuelve un entero que representa el valor Unicode del carácter leído o -1 si se alcanza el final del archivo
<code>void newline()</code>	Escribe una línea de separación
<code>void write(int c)</code>	Escribe un único carácter en el archivo
<code>void write(String str)</code>	Escribe una cadena de caracteres en el archivo
<code>void flush()</code>	Vacía el stream. Si el stream ha guardado cualquier carácter del método <code>write()</code> en un búfer, escribe esos caracteres antes de vaciar el búfer

8. Lectura con BufferedReader

. BufferedReader se utiliza para leer texto de manera eficiente. Internamente, utiliza un búfer (una región de memoria temporal) para almacenar datos de entrada. Al leer, BufferedReader intentará leer tantos caracteres como sea posible a la vez en el búfer, lo que puede ser mucho más eficiente que leer un carácter a la vez. BufferedReader es especialmente útil para leer texto línea a línea mediante `readLine()`

```
try (BufferedReader br =
    new BufferedReader(new FileReader("alumnos.txt"))) {

    String linea;

    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }

} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

Cuando `readLine()` devuelve null, no quedan más líneas por leer.

Ejercicio práctico BufferedWriter

Crea un método que reciba un archivo y devuelva la cantidad de palabras de dicho archivo. Utiliza un buffer para realizar la lectura de forma más eficiente.

9. BufferedWriter

BufferedWriter añade un búfer de memoria. Esto evita escribir físicamente en disco carácter a carácter y suele ser más eficiente. Se utiliza para escribir texto de manera eficiente. Cuando escribes en BufferedWriter, los datos se escriben primero en el buffer y se vacían (se escriben) cuando el buffer está lleno o cuando se usa el método `flush()`. También proporciona el método `newLine()`.

```
try (BufferedWriter bw =
    new BufferedWriter(new FileWriter("alumnos.txt"))) {

    bw.write("1;Ana;DAM");
    bw.newLine();
    bw.write("2;Pedro;DAW");

} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

Aquí aparece try-with-resources. Al terminar el bloque try, Java cierra automáticamente el recurso.

Ejercicio práctico BufferedWriter

Crea un método que reciba el nombre de un archivo y un entero. El método deberá crear el archivo y escribir “n” líneas dentro. Cada línea deberá tener escrito “Esta es la línea n”, sustituyendo “n” por el número de la línea. Utiliza un buffer para realizar la escritura de forma más eficiente.

10. try-with-resources

Es la forma recomendada de trabajar con recursos que deben cerrarse, como ficheros, conexiones o streams.

```
try (BufferedWriter bw =
    new BufferedWriter(new FileWriter("datos.txt"))) {

    bw.write("Texto");

} catch (IOException e) {
    System.out.println(e.getMessage());
}
```

Ventaja principal: aunque ocurra una excepción, Java intenta cerrar el recurso correctamente.

11. De una línea de texto a datos separados

En muchos ficheros cada línea contiene varios campos separados por un delimitador. Podemos utilizar split() para separarlos.

```
String linea = "1;Ana;DAM";

String[] campos = linea.split(";");

int id = Integer.parseInt(campos[0]);
String nombre = campos[1];
String ciclo = campos[2];
```

Esto permite transformar una línea de texto en variables Java y, posteriormente, en un objeto.

```
Alumno alumno = new Alumno(id, nombre, ciclo);
```

12. CSV

CSV significa Comma-Separated Values. Es un formato de texto para representar datos tabulares. Aunque su nombre indica 'separado por comas', en muchos entornos europeos se utiliza punto y coma.

```
id;nombre;edad;ciclo
1;Ana;20;DAM
2;Pedro;21;DAW
3;Lucía;22;DAM
```

Para ficheros CSV sencillos se puede trabajar con `BufferedReader` y `split()`. En CSV reales pueden aparecer campos entrecomillados, separadores dentro del contenido y saltos de línea; en esos casos es preferible usar una librería específica.

13. Leer un CSV sencillo

```
try (BufferedReader br =
    new BufferedReader(new FileReader("alumnos.csv"))) {

    String linea;

    // Saltamos la cabecera
    br.readLine();

    while ((linea = br.readLine()) != null) {

        String[] campos = linea.split(";");

        int id = Integer.parseInt(campos[0]);
        String nombre = campos[1];
        int edad = Integer.parseInt(campos[2]);
        String ciclo = campos[3];

        System.out.println(nombre + " - " + ciclo);
    }
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

14. Escribir un CSV sencillo

```
try (BufferedWriter bw =
    new BufferedWriter(new FileWriter("alumnos.csv"))) {

    bw.write("id;nombre;edad;ciclo");
    bw.newLine();

    bw.write("1;Ana;20;DAM");
    bw.newLine();

    bw.write("2;Pedro;21;DAW");

} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

15. Codificación de caracteres

Un fichero de texto necesita una codificación que indique cómo se representan los caracteres en bytes. UTF-8 es actualmente una de las opciones más habituales y permite representar correctamente tildes, ñ y caracteres de muchos idiomas.

La API Files permite indicar explícitamente la codificación:

```
Path ruta = Path.of("datos.txt");

Files.writeString(
    ruta,
    "Información: canción, año, España",
    StandardCharsets.UTF_8
);
```

Si un fichero se escribe con una codificación y se lee con otra, pueden aparecer caracteres extraños.

16. API moderna: Files.readString y Files.writeString

Para ficheros pequeños, Java ofrece métodos muy directos.

```
Path ruta = Path.of("mensaje.txt");

Files.writeString(ruta, "Hola mundo");

String contenido = Files.readString(ruta);

System.out.println(contenido);
```

Son cómodos cuando el fichero completo cabe razonablemente en memoria. Para ficheros grandes es preferible procesar el contenido progresivamente.

17. Files.readAllLines y Files.write

```
Path ruta = Path.of("alumnos.txt");

List<String> lineas = Files.readAllLines(ruta);

for (String linea : lineas) {
    System.out.println(linea);
}
```

readAllLines() carga todas las líneas en memoria, por lo que tampoco es la mejor opción para ficheros muy grandes.

18. Errores habituales

- No cerrar el fichero.
- Utilizar una ruta incorrecta.

- Sobrescribir un fichero cuando se quería añadir información.
- Olvidar gestionar IOException.
- Suponer que todas las líneas tienen el número correcto de campos.
- Convertir un dato con Integer.parseInt() sin comprobar su formato.
- Usar split() con un separador que puede aparecer dentro de los datos.
- No controlar la codificación del fichero.

19. Ejemplo completo: cargar alumnos desde un CSV

```
import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class Main {

    public static void main(String[] args) {

        List<Alumno> alumnos = new ArrayList<>();

        try (BufferedReader br =
            new BufferedReader(new FileReader("alumnos.csv"))) {

            String linea;
            br.readLine(); // cabecera

            while ((linea = br.readLine()) != null) {

                String[] campos = linea.split(";");

                int id = Integer.parseInt(campos[0]);
                String nombre = campos[1];
                int edad = Integer.parseInt(campos[2]);

                alumnos.add(new Alumno(id, nombre, edad));
            }

        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }

        for (Alumno a : alumnos) {
            System.out.println(a);
        }
    }
}
```

Este ejemplo conecta la persistencia en ficheros con la programación orientada a objetos: los datos almacenados se convierten en objetos Java.

20. Qué debes saber al finalizar la semana

- Diferenciar lectura y escritura.

- Utilizar FileReader y FileWriter.
- Utilizar BufferedReader y BufferedWriter.
- Comprender try-with-resources.
- Leer un fichero línea a línea.
- Escribir y añadir contenido.
- Procesar un fichero CSV sencillo.
- Transformar texto en tipos Java.
- Trabajar con UTF-8.
- Conocer las alternativas modernas de Files.

Solución posible al ejercicio propuesto para BufferedReader

```
public static void contarPalabras(String nombreArchivo) {
    try {

        int palabras = 0;
        FileReader fr = new FileReader(nombreArchivo);
        BufferedReader br = new BufferedReader(fr);
        String linea;

        while ((linea = br.readLine()) != null) {
            String[] palabrasLinea = linea.split("\\s+");
            palabras += palabrasLinea.length;
        }
        //al cerrar el BufferedReader se cierra también el FileReader
        br.close();
        System.out.println(nombreArchivo + " contiene " + palabras + " palabras");

    } catch (IOException e) {
        System.out.println("Error al leer el archivo: " + e.getMessage());
    }
}
```

Solución posible al ejercicio propuesto para BufferedWriter

```
public static void crearLineas(String nombreFichero, int numLineas) {
    try {
        FileWriter fw = new FileWriter(nombreFichero);
        BufferedWriter bw = new BufferedWriter(fw);

        for (int i = 1; i <= numLineas; i++) {
            bw.write("Esta es la línea " + i);
            bw.newLine();
        }
        //al cerrar el BufferedWriter se cierra también el FileWriter
        bw.close();
        System.out.println(nombreFichero + " creado con " + numLineas + " líneas");

    } catch (IOException e) {
        System.out.println("Error al crear o escribir en el archivo: " + e.getMessage());
    }
}
```